

Informe de Laboratorio 09

Tema: JSON Web Tokens (JWT) en Django REST Framework

Nota

Estudiantes	Escuela	Asignatura
Fredy José Aragón Carpio Diego Marcelo Arce Coaquira Gustavo Alonso Carrillo Villalta José Manuel Bravo Rojas	Escuela Profesional de Ingeniería de Sistemas	Desarrollo de Aplicaciones Web Semestre: III Código: 2502117 Grupo: Desarrolladores

Laboratorio	Tema	Duración
09	JSON Web Tokens (JWT) en Django REST Framework	04 horas

Semestre académico	Fecha de inicio	Fecha de entrega
2026 - A	01 de Junio de 2026	09 de Junio de 2026

Índice

1. Laboratorio 09: Autenticación con JWT	3
1.1. Descripción y Objetivos	3
1.2. Entregables	3
1.3. Entorno de Implementación	3
2. Equipos, materiales y temas utilizados	3
3. URL del Repositorio en GitHub	4
4. Desarrollo de actividades	4
4.1. Estructura del proyecto	4
4.2. Instalación del entorno virtual y dependencias	5
4.3. Instalación y Configuración de JWT	5
4.4. Registro de EndPoints en urls.py	6
4.5. JSON Anidados Estratégicos	6

4.5.1. Justificación Técnica y Explicación	6
4.5.2. Implementación de Serializadores Anidados	6
4.5.3. Ejemplo de Respuesta JSON Anidada	8
4.6. Pruebas con Cliente REST (Postman)	9
4.6.1. Generación de Token — POST /api/token/	9
4.6.2. Rechazo sin Token — HTTP 401 Unauthorized	10
4.6.3. GET — Lectura Protegida con Token	12
4.6.4. POST — Creación de Registro Autorizado	13
4.6.5. PUT — Modificación de Registro Autorizado	14
4.6.6. DELETE — Eliminación de Registro Autorizado	15
4.7. Deploy en plataforma en la nube	16
5. Resultados	16
6. Autoevaluación	17
7. Rúbrica de Calificación	17
8. Referencias	17

1. Laboratorio 09: Autenticación con JWT

En este laboratorio se integró el mecanismo de seguridad y autenticación mediante **JSON Web Tokens (JWT)** sobre el ecosistema de Django REST Framework utilizando el paquete `django-rest-framework-simplejwt`. A diferencia de la práctica anterior, todas las operaciones críticas sobre los recursos (GET, POST, PUT, DELETE) han sido restringidas, requiriendo un token de acceso válido provisto en las cabeceras HTTP.

Asimismo, se diseñaron estructuras de datos complejas mediante la creación de JSON anidados estratégicos para optimizar el rendimiento de las consultas y se mantuvieron las buenas prácticas de despliegue continuo.

1.1. Descripción y Objetivos

Esta práctica se enfocó en asegurar el acceso a los EndPoints de una API RESTful. Los objetivos principales fueron:

- Instalar, configurar y personalizar **Simple JWT** en las configuraciones globales del proyecto Django.
- Implementar la lógica de generación y refresco de tokens mediante rutas dedicadas en `urls.py`.
- Proteger las operaciones CRUD (GET, POST, PUT, DELETE) para que solo usuarios autenticados realicen peticiones.
- Diseñar y justificar técnicamente el uso de **JSON anidados estratégicos** en consultas de alta complejidad relacional.
- Validar el comportamiento de los endpoints protegidos usando un cliente REST (Postman), capturando evidencias con y sin cabeceras de autorización.
- Actualizar la documentación técnica en el archivo `README.md` y asegurar el despliegue del sistema en producción.

1.2. Entregables

- **Repositorio:** <https://github.com/FredyAragon/Cafe-Sys>
- **Video de Demostración (CRUD con Token):** <https://youtu.be/NWct8qIU3ps>
- **Informe PDF:** <https://github.com/FredyAragon/Cafe-Sys/blob/main/informes/Informe-Lab9-DAW.pdf>
- **Deploy en Render:** <https://cafesys-backend.onrender.com>

1.3. Entorno de Implementación

- Python 3.10+ e instalador de paquetes `pip`.
- Django y Django REST Framework.
- **django-rest-framework-simplejwt 5.3+**.
- Entorno virtual de Python (`venv`).
- Postman (Cliente de pruebas REST).
- Git y GitHub para control de versiones.

2. Equipos, materiales y temas utilizados

- Entorno Virtualizado en Python.
- Autenticación basada en Tokens vs Autenticación por Sesión tradicional.

- Arquitectura JWT: Cabecera (Header), Carga útil (Payload) y Firma (Signature).
- Serializadores Anidados Relacionales en DRF.
- Postman para la inyección de Tokens en el Header **Authorization: Bearer <token>**.

3. URL del Repositorio en GitHub

- **Repositorio Oficial:** <https://github.com/FredyAragon/Cafe-Sys>
- **Informe en Repositorio:** <https://github.com/FredyAragon/Cafe-Sys/blob/main/informes/Informe-Lab9-DAW.pdf>

4. Desarrollo de actividades

4.1. Estructura del proyecto

Se detalla la disposición del módulo de configuración y las aplicaciones modificadas para dar soporte a JWT:

```
cafesys-project/
|
+-- backend/                                <- Entorno del Servidor (Django)
| |
| | +-- apps/
| | | +-- core/                            <- Aplicacion Principal del Sistema
| | |   +-- migrations/                   <- Historial de Migraciones a Supabase
| | |   +-- models/                      <- Definicion de Tablas (Ej. Product)
| | |   +-- admin.py                     <- Registro en Panel de Administracion
| | |   +-- apps.py                      <- Configuracion de la App
| | |   +-- serializers.py               <- Conversion de Modelos a JSON
| | |   +-- urls.py                      <- Enrutamiento de la API
| | |   +-- views.py                    <- Logica de Controladores/Endpoints
| | |
| | +-- cafesys/                          <- Configuracion Global de Django
| | | +-- settings.py                    <- Ajustes Generales
| | | +-- urls.py                       <- Enrutador Raiz del Proyecto
| | | +-- asgi.py
| | | +-- wsgi.py
| | |
| | +-- venv/                            <- Entorno Virtual Local (Python)
| | +-- .env                             <- Variables de Entorno Secretas
| | +-- .gitignore
| | +-- Dockerfile                       <- Configuracion del Contenedor Backend
| | +-- manage.py                        <- Gestor de Comandos de Django
| | +-- README.md
| | +-- requirements.txt                 <- Dependencias (django-cors-headers, etc.)
| |
+-- bd/                                   <- Base de Datos y Modelado
| | +-- cafesysdb.sql                   <- Script de Respaldo SQL
| | +-- DER-dgdiagram.io.png            <- Diagrama Entidad-Relacion (Web)
| | +-- DER-Resumen.png                 <- Imagen de Resumen del Modelo BD
| |
+-- frontend/                             <- Entorno de Cliente (Angular v21)
| | +-- .gitignore
```

```
|  
+-- informes/                <- Documentacion Academica UNSA  
|   +-- ...                  <- Archivos LaTeX / Informes de Lab  
|  
+-- docker-compose.yml       <- Orquestador de Contenedores Locales  
+-- render.yaml              <- Configuracion para Deploy en Render  
+-- README.md                <- Documentacion General del Repositorio
```

4.2. Instalación del entorno virtual y dependencias

Antes de instalar cualquier paquete, se debe activar el entorno virtual del proyecto:

Listing 1: Creación y activación del entorno virtual.

```
python -m venv venv  
source venv/bin/activate      # Linux / Mac  
# venv\Scripts\activate       # Windows  
  
pip install django djangorestframework  
pip install djangorestframework-simplejwt  
pip freeze > requirements.txt
```

4.3. Instalación y Configuración de JWT

El paquete `djangorestframework-simplejwt` provee un backend de autenticación JWT listo para DRF. A continuación se detalla su configuración en `settings.py`:

Listing 2: Modificación de `REST_FRAMEWORK` en `settings.py`.

```
# settings.py  
  
REST_FRAMEWORK = {  
    'DEFAULT_AUTHENTICATION_CLASSES': [  
        'rest_framework_simplejwt.authentication.JWTAuthentication',  
    ],  
    'DEFAULT_PERMISSION_CLASSES': [  
        'rest_framework.permissions.AllowAny',  
    ],  
    'DEFAULT_RENDERER_CLASSES': [  
        'rest_framework.renderers.JSONRenderer',  
        'rest_framework.renderers.BrowsableAPIRenderer',  
    ],  
}  
  
SIMPLE_JWT = {  
    'ACCESS_TOKEN_LIFETIME': timedelta(minutes=60), # El token de acceso dura 1 hora  
    'REFRESH_TOKEN_LIFETIME': timedelta(days=1), # El token de refresco dura 1 día  
    'ROTATE_REFRESH_TOKENS': False,  
    'BLACKLIST_AFTER_ROTATION': False,  
    'UPDATE_LAST_LOGIN': False,  
  
    'ALGORITHM': 'HS256',  
    'SIGNING_KEY': SECRET_KEY, # Usa la misma clave secreta de tu Django  
    'AUTH_HEADER_TYPES': ('Bearer',), # En Postman usaremos: Bearer <token>  
    'AUTH_TOKEN_CLASSES': ('rest_framework_simplejwt.tokens.AccessToken',),  
    'USER_ID_FIELD': 'id',
```

```
'USER_ID_CLAIM': 'user_id',  
}
```

4.4. Registro de EndPoints en urls.py

Para exponer los endpoints de obtención y refresco de tokens, se añaden las vistas nativas de Simple JWT al enrutador principal del proyecto:

Listing 3: Rutas de autenticación JWT y negocio en urls.py.

```
# config/urls.py  
  
from django.urls import path, include  
from rest_framework_simplejwt.views import (  
    TokenObtainPairView,  
    TokenRefreshView,  
)  
  
# URLs de la aplicación Core  
urlpatterns = [  
    # Login adaptado para desarrollo local (Usa tu Custom Serializer)  
    path('token/', CustomTokenObtainPairView.as_view(), name='token_obtain_pair'),  
  
    # NUEVO: Refresco de token para mantener la sesión en Angular  
    path('token/refresh/', TokenRefreshView.as_view(), name='token_refresh'),  
  
    # Endpoints del Router  
    path('', include(router.urls)),  
]
```

4.5. JSON Anidados Estratégicos

4.5.1. Justificación Técnica y Explicación

La creación de JSON anidados responde a la necesidad de disminuir el número de peticiones HTTP (*Round-trips*) del cliente hacia el servidor. En lugar de solicitar un objeto base y realizar subsiguientes llamadas para obtener sus llaves foráneas, un serializador anidado estratégico resuelve y agrupa la información jerárquica en una sola transacción estructurada.

4.5.2. Implementación de Serializadores Anidados

Listing 4: Serializadores anidados en apps/core/serializers.py.

```
# apps/core/serializers.py  
  
from rest_framework import serializers  
from rest_framework_simplejwt.serializers import TokenObtainPairSerializer  
from rest_framework import exceptions  
from django.contrib.auth import get_user_model  
  
class OrderDetailsSerializer(serializers.ModelSerializer):  
    product_name = serializers.CharField(source='product.name', read_only=True)  
  
    class Meta:  
        model = OrderDetails  
        fields = (
```

```
        'id', 'order', 'product', 'product_name',
        'quantity', 'unitPrice', 'subtotal',
        'status', 'created', 'modified'
    )
    read_only_fields = ('id', 'subtotal', 'product_name', 'created', 'modified')

class ProductDetailNestedSerializer(serializers.ModelSerializer):
    class Meta:
        model = Products
        fields = ('id', 'name', 'price', 'imageUrl', 'status')

class OrderDetailsNestedSerializer(serializers.ModelSerializer):
    product_detail = ProductDetailNestedSerializer(source='product', read_only=True)

    class Meta:
        model = OrderDetails
        fields = (
            'id', 'product', 'product_detail',
            'quantity', 'unitPrice', 'subtotal', 'status'
        )
        read_only_fields = ('id', 'subtotal', 'product_detail')

class OrdersSerializer(serializers.ModelSerializer):
    details = OrderDetailsNestedSerializer(many=True, read_only=True)
    user_detail = UsersSerializer(source='user', read_only=True)

    class Meta:
        model = Orders
        fields = (
            'id', 'user', 'user_detail', 'location', 'orderStatus',
            'total', 'notes', 'details', 'status', 'created', 'modified'
        )
        read_only_fields = ('id', 'total', 'details', 'user_detail', 'created', 'modified')
```

Listing 5: Vista protegida con permiso IsAuthenticated en api/views.py.

```
# api/views.py

from rest_framework import viewsets, filters, permissions
from rest_framework.permissions import IsAuthenticated, AllowAny
from rest_framework_simplejwt.views import TokenObtainPairView
from .serializers import CustomTokenObtainPairSerializer

class OrdersViewSet(viewsets.ModelViewSet):
    permission_classes = [IsAuthenticated]
    queryset = Orders.objects.select_related(
        'user', 'location'
    ).prefetch_related(
        'details', 'details__product'
    ).all()
    serializer_class = OrdersSerializer
    filter_backends = [filters.SearchFilter, filters.OrderingFilter]
    search_fields = ['user__firstName', 'user__lastName', 'user__email']
```

```
ordering_fields = ['created', 'total', 'orderStatus']  
ordering        = ['-created']
```

4.5.3. Ejemplo de Respuesta JSON Anidada

Listing 6: Payload JSON estructurado resultante del endpoint de matrículas.

```
[  
  {  
    "id": 2,  
    "user": 8,  
    "user_detail": {  
      "id": 8,  
      "firstName": "Pedrito",  
      "lastName": "Juarez",  
      "email": "pedrito.123@gmail.com",  
      "role": 16,  
      "status": "active",  
      "created": "2026-06-04T18:55:39.394238-05:00",  
      "modified": "2026-06-04T18:55:39.394244-05:00"  
    },  
    "location": 2,  
    "orderStatus": "delivered",  
    "total": "8.50",  
    "notes": "Cafe Americano",  
    "details": [],  
    "status": "active",  
    "created": "2026-06-04T18:57:16.789973-05:00",  
    "modified": "2026-06-04T18:57:16.789990-05:00"  
  },  
  {  
    "id": 1,  
    "user": 7,  
    "user_detail": {  
      "id": 7,  
      "firstName": "Juanito",  
      "lastName": "Perez",  
      "email": "juanito@gmail.com",  
      "role": 16,  
      "status": "active",  
      "created": "2026-06-04T18:46:11.414376-05:00",  
      "modified": "2026-06-04T18:46:11.414385-05:00"  
    },  
    "location": 1,  
    "orderStatus": "pending",  
    "total": "12.00",  
    "notes": "Deseo un Cafe Latte bien caliente y cargado",  
    "details": [  
      {  
        "id": 1,  
        "product": 9,  
        "product_detail": {  
          "id": 9,  
          "name": "Café Latte",  
          "price": "12.00",
```



```
        "imageUrl": "https://cdn7.kiwilimon.com/recetaimagen/36986/640x640/46349.jpg.jpg",
        "status": "active"
    },
    "quantity": 1,
    "unitPrice": "12.00",
    "subtotal": "12.00",
    "status": "active"
  }
],
"status": "active",
"created": "2026-06-04T18:47:39.410753-05:00",
"modified": "2026-06-04T18:54:10.748521-05:00"
}
```

4.6. Pruebas con Cliente REST (Postman)

Se realizaron pruebas de inyección de métodos HTTP para constatar que los decoradores y configuraciones de seguridad bloquean los accesos no autorizados y responden positivamente ante el uso de la cabecera **Authorization: Bearer <token>**.

4.6.1. Generación de Token — POST /api/token/

Se envía una petición POST con las credenciales del usuario (**username** y **password**). El servidor responde entregando un **access** token (vigencia 60 min) y un **refresh** token (vigencia 1 día).

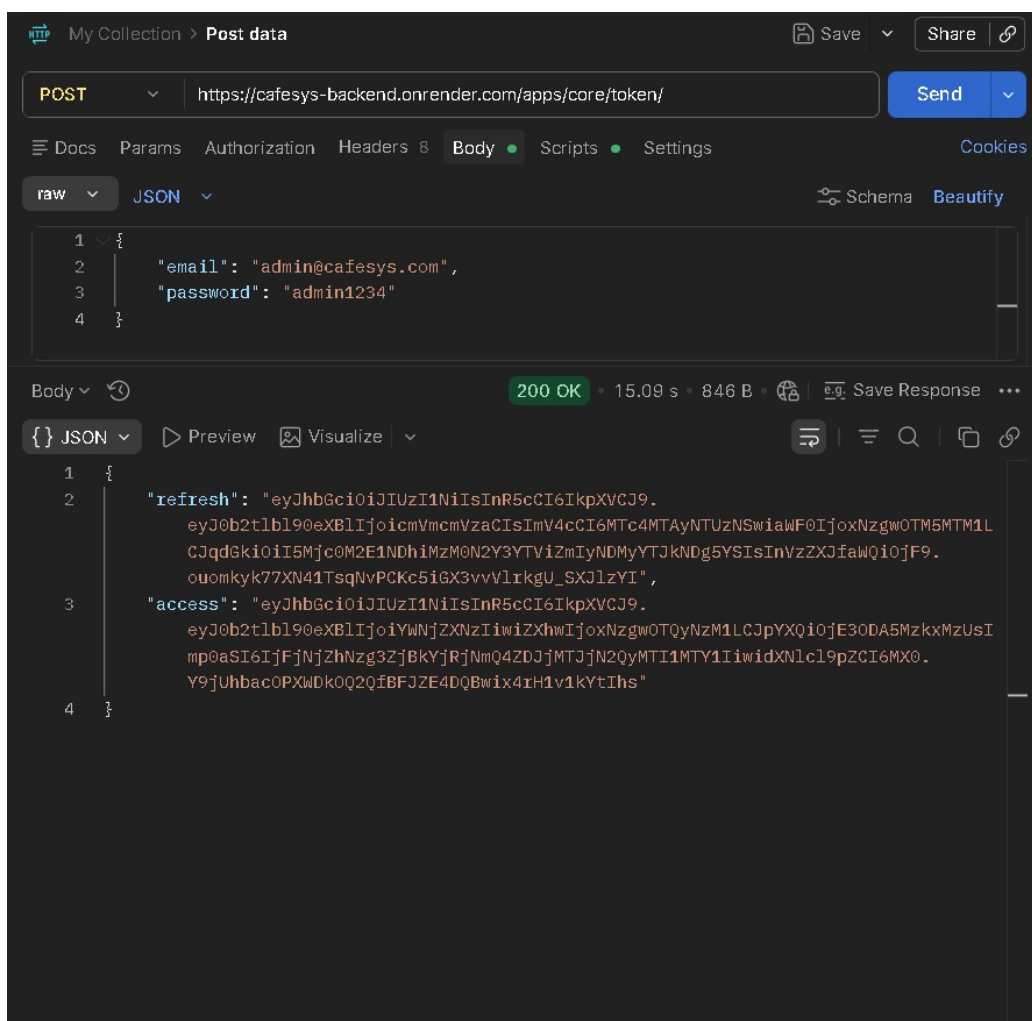


Figura 1: Generación del par de tokens JWT mediante Postman.

4.6.2. Rechazo sin Token — HTTP 401 Unauthorized

Antes de mostrar las operaciones exitosas, se evidencia el comportamiento de la API ante una petición sin cabecera de autorización.

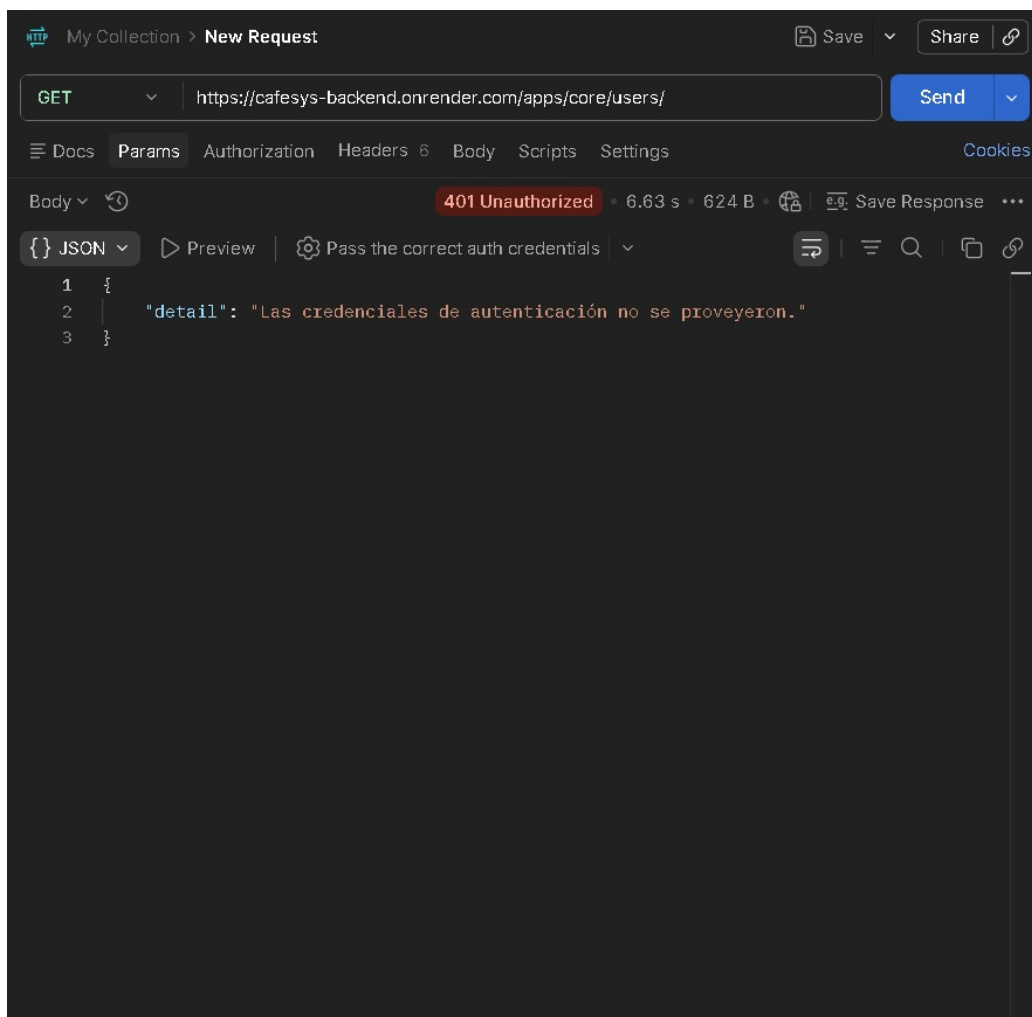


Figura 2: Denegación automática (HTTP 401) ante acceso anónimo.

4.6.3. GET — Lectura Protegida con Token

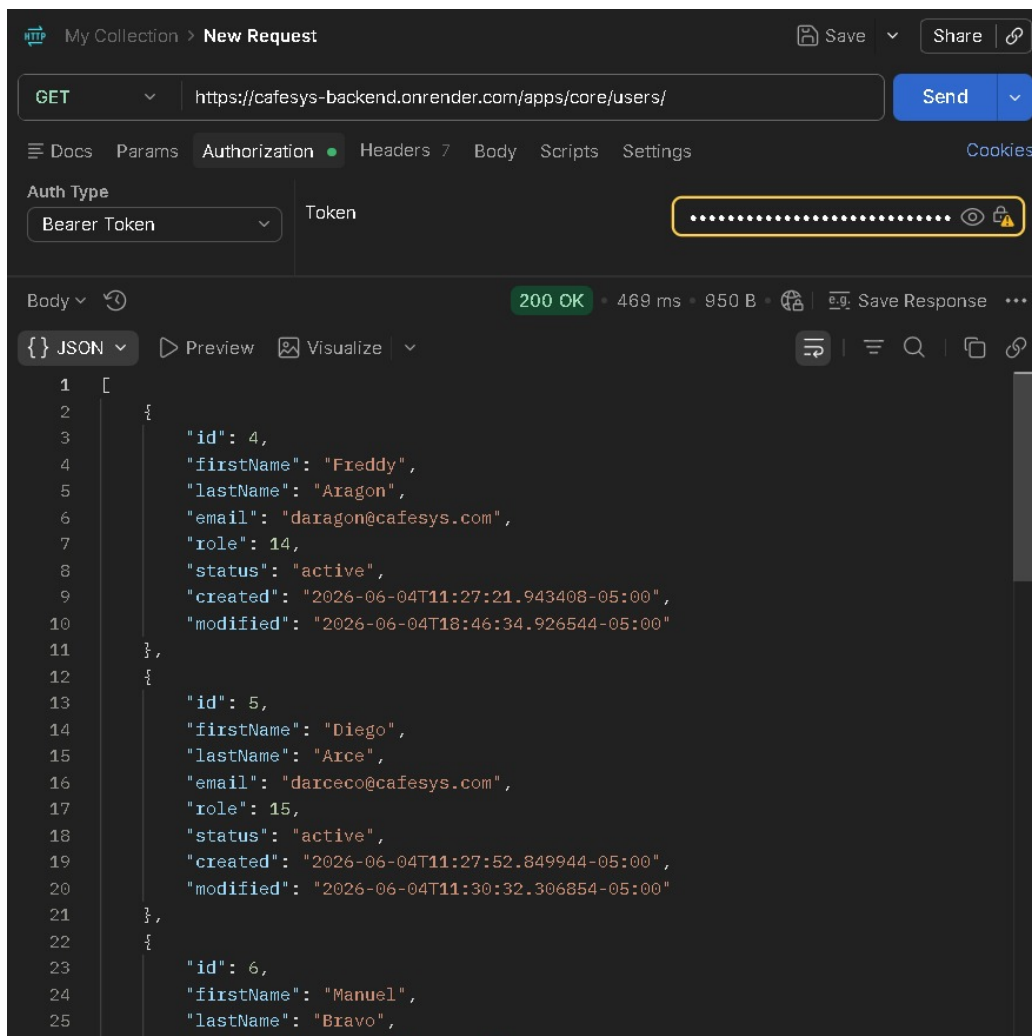


Figura 3: GET authorized con token JWT — lista de matrículas anidadas.

4.6.4. POST — Creación de Registro Autorizado

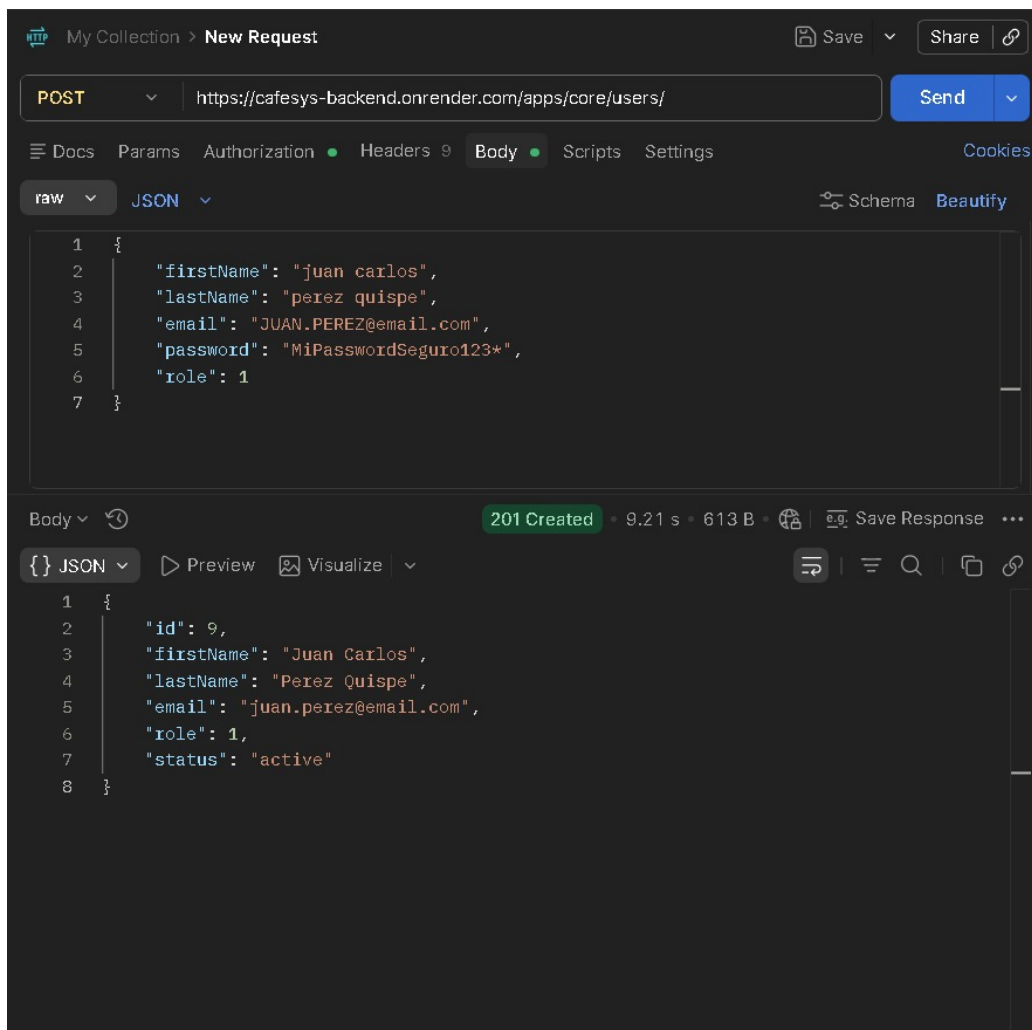


Figura 4: POST autorizado — creación de un nuevo registro.

4.6.5. PUT — Modificación de Registro Autorizado

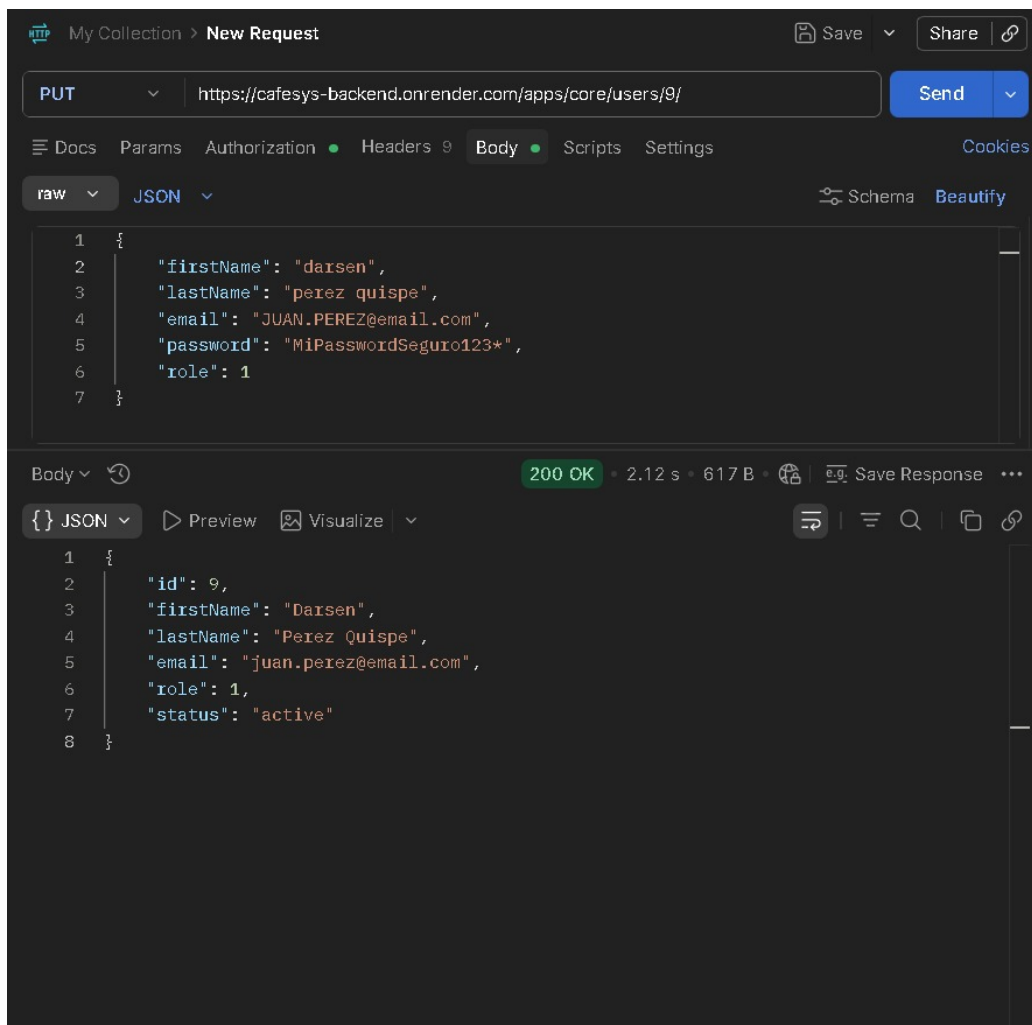


Figura 5: PUT autorizado — modificación de un registro existente.

4.6.6. DELETE — Eliminación de Registro Autorizado

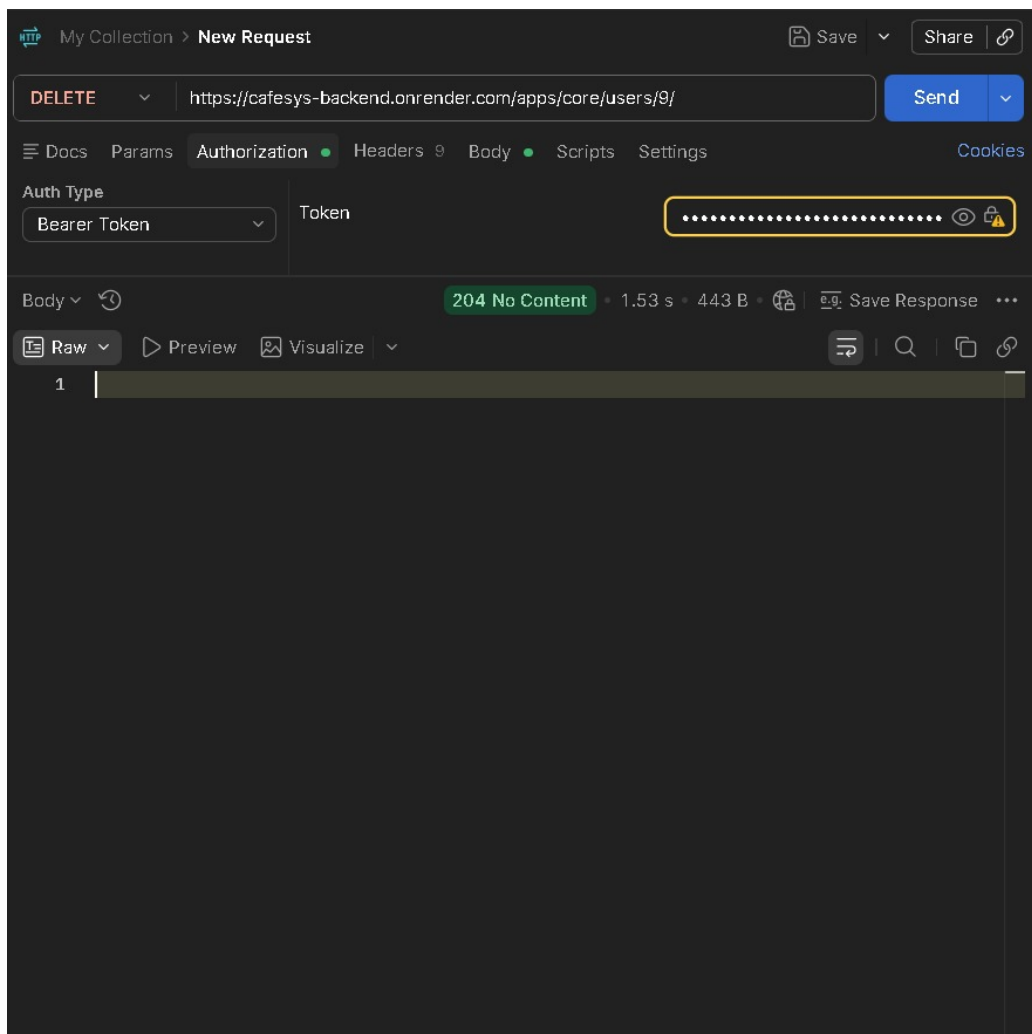


Figura 6: DELETE autorizado — eliminación del registro indicado por ID.

4.7. Deploy en plataforma en la nube

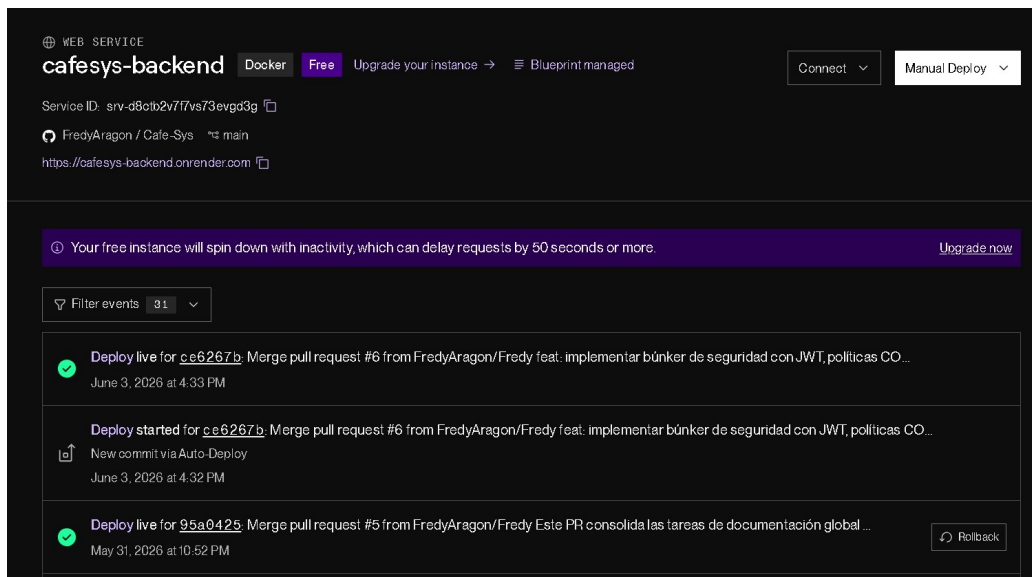


Figura 7: Proyecto desplegado en plataforma en la nube.

5. Resultados

- Se implementó de manera satisfactoria el protocolo JWT, logrando una denegación automática (HTTP 401 Unauthorized) ante cualquier intento de operación CRUD anónima.
- Los JSON anidados estratégicos optimizaron las consultas complejas, unificando los datos del estudiante y los cursos en la respuesta del endpoint de matrículas, reduciendo el número de round-trips.
- Todas las operaciones CRUD (GET, POST, PUT, DELETE) funcionan correctamente cuando se inyecta el token de acceso en la cabecera **Authorization: Bearer <token>**.
- El proyecto fue desplegado exitosamente en una plataforma en la nube y se encuentra accesible mediante URL pública.

6. Autoevaluación

Nombre	Rol	Rama	Tareas Realizadas	% Trabajo
Aragón Carpio Fredy José	Backend	Fredy	Instalación y configuración de JWT	100 %
Arce Coaquira Diego Marcelo	Frontend	Diego	Pruebas con cliente REST (Postman)	100 %
Bravo Rojas José Manuel	FullStack	Jose	Elaboración del informe y README.md	100 %
Carrillo Villalta Gustavo Alonso	Base de Datos	Gustavo	Elaborar Jsons Anidados	100 %
Total				100 %

Cuadro 1: Desglose de actividades, roles y autoevaluación.

7. Rúbrica de Calificación

Ítem	Descripción	Puntaje
Entorno Virtual	Uso correcto del entorno virtual en proyectos de Python.	1
Instalación y config. JWT	Realiza detalladamente los pasos para instalar y configurar JWT.	4
JSON anidados	Crea Serializadores y Vistas para producir JSON anidados estratégicos para consultas complejas.	3
Informe	El laboratorio tiene un informe que detalla todos los pasos necesarios para el desarrollo de la práctica.	5
Deploy	El laboratorio está deployado en una plataforma en la nube.	5
Prueba ¹	Se tomaron en cuenta todas las consideraciones y recomendaciones, lo que evidencia un trabajo en equipo.	-0
Total		18

Cuadro 2: Rúbrica oficial del Laboratorio 09.

8. Referencias

- <https://django-rest-framework-simplejwt.readthedocs.io/en/latest/>
- <https://jwt.io/introduction/>
- <https://www.django-rest-framework.org/api-guide/permissions/#isauthenticated>

¹El docente debe comprobar el cumplimiento de todas las consideraciones y recomendaciones, evidenciando trabajo en equipo con responsabilidad y práctica de la ética profesional, a fin de no aplicar ninguna penalidad.

- <https://www.django-rest-framework.org/api-guide/authentication/>
- <https://www.geeksforgeeks.org/python/jwt-authentication-with-django-rest-framework/>